

Matrix Chain Multiplication

Matrix i : $p_i \times p_{i+1}$. $dp[i][j]$ = min cost for $A_i \cdots A_j$.
 $dp[i][i] = 0, \quad dp[i][j] = \min_{i \leq k < j} \{ dp[i][k] + dp[k+1][j] + p_i p_{k+1} p_{j+1} \}$

```
fn matrix_chain(p:Vec) { // optimal parenthesization
    n:=p.len()-1; dp:=vec![vec![0;n];n]; split:=vec![vec![0;n];n];
    for len in 2..=n {for i in 0..=n-len {
        j:=i+len-1; dp[i][j]=INF;
        for k in i..j {
            cost:=dp[i][k]+dp[k+1][j]+p[i]*p[k+1]*p[j+1];
            if cost<dp[i][j] {dp[i][j]=cost; split[i][j]=k}
        }
    }
    }
    (dp,split)
}
```

Increasing chain length. $\Theta(n^2)$ states $\times \Theta(n)$ splits = $\Theta(n^3)$.

Optimal BST

Sorted keys K_i with successful probs p_i ; optional unsuccessful q_i . $e[i][j]$ = min expected cost for subtree $i..j$, $w[i][j]$ = prob mass, $root[i][j]$ = best root.

$e[i][i-1] = 0, \quad w[i][i-1] = q_{i-1},$
 $e[i][j] = \min_{i \leq r \leq j} \{ e[i][r-1] + e[r+1][j] + w[i][j] \}$

```
fn optimal_bst(p,q,n) {
    for i in 1..=n+1 {e[i][i-1]=0; w[i][i-1]=q[i-1]}
    for len in 2..=n {for i in 1..=n-len+1 {
        j:=i+len-1; e[i][j]=INF; w[i][j]=w[i][i-1]+p[i]+q[j+1];
        for r in i..=j {
            cost:=e[i][r-1]+e[r+1][j]+w[i][j];
            if cost<e[i][j] {e[i][j]=cost; root[i][j]=r}
        }
    }
    }
    (e,root)
}
```

Runtime $\Theta(n^3)$, space $\Theta(n^2)$.

Longest Common Subsequence

$dp[i][j]$ = LCS length of $A[0..i], B[0..j]$.
 $dp[0][j] = dp[i][0] = 0,$
 $dp[i][j] = \begin{cases} dp[i-1][j-1] & A[i-1] = B[j-1] \\ \max(dp[i-1][j], dp[i][j-1]) & A[i-1] \neq B[j-1] \end{cases}$

```
fn lcs(a:Vec<char>, b:Vec<char>) {
    n:=a.len(); m:=b.len(); dp:=vec![vec![0;m+1];n+1];
    for i in 1..=n {for j in 1..=m {
        dp[i][j]=if a[i-1]==b[j-1] {dp[i-1][j-1]+1}
        else {dp[i-1][j].max(dp[i][j-1])}
    }
    }
    dp[n][m]
}
```

```
fn lcs_reconstruct(a:Vec<char>, b:Vec<char>, dp:Vec<Vec<usize>>){
    i:=a.len(); j:=b.len(); ans:=vec![];
    while i>0 && j>0 {
        if a[i-1]==b[j-1] {ans.push(a[i-1]); i--; j--;}
        else if dp[i-1][j]>dp[i][j-1] {i--;}
        else {j--;}
    }
    ans.reverse(); ans
}
```

$\Theta(nm)$ time, $\Theta(nm)$ or rolling $\Theta(m)$ space. Equal $\rightarrow$ diagonal+1; unequal $\rightarrow$ max(up,left).

Knapsack 0/1

$dp[i][c]$ = best value using first i items, capacity c .
 $dp[i][c] = \max \{ dp[i-1][c], v_{i-1} + dp[i-1][c-w_{i-1}] \} \quad (w_{i-1} \leq c).$

```
fn knapsack(w:Vec<usize>, v:Vec<i32>, cap:usize) { // choose item or not once
    n:=w.len(); dp:=vec![vec![0;cap+1];n+1];
    for i in 1..=n {for c in 0..=cap {
        dp[i][c]=dp[i-1][c];
        if w[i-1]<=c {
            dp[i][c]=dp[i-1][c].max(v[i-1]+dp[i-1][c-w[i-1]])
        }
    }
    }
    dp[n][cap]
}
```

// 1D 0/1: capacity descending. Unbounded: capacity ascending.

$\Theta(nW)$ pseudo-polynomial.

Coin Change / Change-Making

$mincoins[0] = 0, \quad mincoins[x] = \min_{coin \leq x} \{ 1 + mincoins[x-coin] \}$

fn coin_change(coins:Vec<usize>, target:usize) { // unlimited coins
 mincoins:=vec![INF;target+1]; mincoins[0]=0; // min number of coins
 for amount in 1..=target {for coin in coins {
 if coin<=amount {
 mincoins[amount]=mincoins[amount].min(1+mincoins[amount-coin])
 }
 }
 }
 mincoins[target]
}

Runtime $\Theta(target \cdot \#coins)$.

Longest Increasing Subsequence

Quadratic:
 $dp[i] = 1 + \max\{ dp[j] : j < i \wedge a_j < a_i \}, \quad \text{answer} = \max_i dp[i].$

```
fn lis_n2(a:Vec<i32>){
    dp:=vec![a.len();0];
    for i in 0..a.len(){for j in 0..i {
        if a[j]<a[i]{dp[i]=dp[j].max(1+dp[j])}
    }
    }
    max(dp)
}

fn lis_nlogn(a:Vec<i32>){ // tails[len-1] = min tail
    tails:=vec![];
    for v in a {
        l:=0; r:=tails.len();
        while l==r {l=(l+r)/2; if tails[l]<= v {l=1+else{r=l}}}
        if l==tails.len(){tails.push(v)}else{tails[l]=v}
    }
    tails.len()
}
```

DAG Longest Path

Input `adj: WGraph`. Topological order + max-relaxation; negative weights OK because no cycles.
fn dag_longest(adj:WGraph, source:usize){
 m:=adj.len(); dist:=vec![INF;n]; parent:=vec![None;n]; dist[source]=0;
 for u in topo(adj) {if dist[u]==INF {for (v,w) in adj[u] {
 if dist[u]+w<dist[v] {dist[v]=dist[u]+w; parent[v]=Some(u)}
 }
 }
 }
 dist
}

Runtime $\Theta(V + E)$.

8. Graph Representation / Types

Use these names everywhere:

```
type Graph = Vec<Vec<usize>>; // adj[u] = neighbors u
type WGraph = Vec<Vec<usize,i32>>; // adj[u] = (v,weight)
struct Edge { u:usize, v:usize, w:i32 }
type Edgelist = Vec<Edge>;

adj list          space  $\Theta(V^2 + E)$       scan neighbors  $\Theta(\deg u)$ 
adj matrix        space  $\Theta(V^2)$           edge lookup  $\Theta(1)$ , scan  $\Theta(V)$ 
edge list         space  $\Theta(E)$            Bellman-Ford/Kruskal scans all edges
```

9. BFS

Input `adj: Graph`. Unweighted shortest paths / levels / components / bipartite. Runtime $\Theta(V + E)$.

```
fn bfs(adj:Graph, source:usize){ // 2-coloring
    n:=adj.len(); // shortest path in unweighted graph
    dist:=vec![INF;n]; // levels, components
    parent:=vec![None;n];
    queue:=VecDeque::new();
    for s in 0..n {
        if color[s]==1 {continue}
        color[s]=0; queue.push_back(s);
        while let Some((u,w))=queue.pop_front(){
            for v in adj[u] {
                while let Some(u)=queue.pop_front(){
                    for v in adj[u] {
                        if color[v]==1 {
                            color[v]=!color[v];
                            queue.push_back(v);
                        } else if color[v]==color[u] {return false}
                    }
                }
            }
        }
    }
    (dist,parent)
}
```

Invariant: queue processed in nondecreasing distance. First discovery gives shortest unweighted distance. If a shorter path to v existed, its predecessor would have been processed earlier, so v would already be discovered.

Uses: unweighted shortest path, levels, connected components, bipartite test.

Path reconstruction: follow parent[i] backward until source, then reverse.

10. DFS / Topological Sort / SCC

Input `adj: Graph`. Colors: WHITE/GRAY/BLACK. Runtime $\Theta(V + E)$.

```
fn dfs(adj:Graph) { // topo, cycle detection, SCC
    n:=adj.len(); color:=vec![WHITE;n]; parent:=vec![None;n]; time=0;
    fn visit(u:usize) {
        time+=1; discover[u]=time; color[u]=GRAY;
        for v in adj[u] {
            if color[v]==WHITE {parent[v]=Some(u); visit(v)}
        }
        color[u]=BLACK; time+=1; finish[u]=time;
        for u in 0..n {if color[u]==WHITE {visit(u)}}
    }
}
```

Directed edges: tree = white target; back = gray ancestor; forward/cross = black. Undirected: only tree/back; cycle test = visited neighbor $\neq$ parent.

$\begin{matrix} d[u] < d[v] & \text{tree/forward} \\ d[u] < f[v] & \text{v descendant of u} \\ f[u] < d[v] & \text{v ancestor, cycle in directed} \\ f[u] < f[v] & \text{disjoint subtrees} \end{matrix}$

Directed cycle detection: edge to GRAY vertex $\Leftrightarrow$ back edge $\Leftrightarrow$ directed cycle. Edge to BLACK does not prove a cycle.

Topological Sort

DAG order: every edge $u \rightarrow v$ has u before v . DFS = decreasing finish times.

```
fn topo_dfs(adj:Graph){
    n:=adj.len(); seen:=vec![false;n]; order:=vec![];
    fn visit(u:usize) {
        seen[u]=true;
        for v in adj[u] {if !seen[v]{visit(v)}}
        order.push(u)
    }
    for u in 0..n {if !seen[u]{visit(u)}}
    order.reverse(); order
}
```

```
fn topo_kahn(adj:Graph){
    n:=adj.len(); indeg:=vec![0;n];
    queue:=VecDeque::new(); order:=vec![];
    for v in 0..n {for v in adj[u]{indeg[v]+=1}}
    while let Some(u)=queue.pop_front(){
        order.push(u);
        for v in adj[u]{
            indeg[v]-=1;
            if indeg[v]==0{queue.push_back(v)}
        }
        if order.len()==n {Some(order)} else {None}
    }
    comps
}
```

Kahn returns `None` iff not all vertices are output.

Strongly Connected Components

SCC: $u \rightsquigarrow v$, Kosaraju = DFS finish order, reverse graph, DFS in decreasing finish order.

```
fn kosaraju(adj:Graph){ // SCC. DFS order -> reverse g
    order:=dfs_finish_order(adj); reverse_graph(adj);
    seen:=vec![false;n]; comps=[];
    for source in order.rev() {
        if !seen[source]{continue}
        comps.push(source);
        for v in adj[source] {
            if !seen[v]{seen[v]=true; stack.push(v)}
        }
        while let Some((du,u))=heap.pop_min(){
            indeg[v]-=1;
            if indeg[v]==0{queue.push_back(v)}
        }
        if order.len()==n {Some(order)} else {None}
    }
    comps
}
```

Complexities:
• linked list weighted union: $O(m + n \log n)$
• forest + union by rank + path compression: $O(m \alpha(n))$
 $\alpha(n) \leq 5$ in practice.

```
//Connected components: //Cycle detection (undirected):
let mut dsu: DSU = DSU::new(n); let mut dsu: DSU = DSU::new(n);
for (u, v) in edges { for (u, v) in edges {
    dsu.union(u, v); // cycle
}
}
if dsu.find(a) == dsu.find(b) {
    // same component
}
```

12. Minimum Spanning Trees

Chooser

dense graph Kruskal often easiest
disconnected graph Prim often easier tree: connected + acyclic connected graph: tree $\Leftrightarrow$ forest, no MST

|E| = V-1
MST: spanning tree with min total edge weight

Cut / Cycle Properties

Cut $(S, V - S)$: edge crosses cut iff one endpoint in S , one outside. Light edge = minimum-weight crossing edge.

Cut property: if cut respects chosen edge A , then a light crossing edge is safe.

Cycle property: for any cycle, a heaviest edge on that cycle is excluded from some MST. If weights distinct, the heaviest edge is in no MST.

Kruskal

Data structures:

```
struct Edge {
    u,
    v,
    w: i32,
}

let mut edges: Vec<Edge> = ...;
let mut dsu: DSU = DSU::new(n);
let mut mst: Vec<Edge> = Vec::new();
let mut total: i32 = 0;

fn kruskal(e: Vec<Edge>){ // MST
    sort_by(|edges|); dsu:=DSU::new(n); total=0; mst=[];
    for e in edges { // sort edges + DSU
        if dsu.union(e.u,e.v) {
            total+=e.w; mst.push(e);
            if mst.len()==n-1 {break}
        }
    }
    (total,mst)
}
```

Complexity: sort: $O(E \log E)$ DSU: $O(E \alpha(V))$ total: $O(E \log E)$
Output condition: if `mst.len()==n-1`, it is an MST; otherwise graph disconnected and result is a forest.

Traps:

- If graph disconnected $\rightarrow$ MST does not exist; returns forest.
- Need sort by weight increasing.
- Skip edge if endpoints already connected.

Prim

```
let mut adj: Vec<Vec<usize, i32>> = vec![]; n:
let mut used: Vec<bool> = vec![false; n];
let mut heap: BinaryHeap<Reverse<i32, usize>> = BinaryHeap::new();
// (weight, vertex, parent)

fn prim(adj,a){ // MST
    heap.push((0,s)); total=0; cnt=0; // min crossing edge
    while let Some((u,w))=heap.pop_min(){
        if used[u] {continue}
        used[u]=true; total+=w; cnt+=1;
        for (v,w2) in adj[u] {
            if !used[v] {heap.push((w2,v))}
        }
        if cnt!=n {/" disconnected "/}
    }
    total
}
```

Complexity:

• Binary heap lazy: $O(E \log E)$; for simple graphs $E \leq V^2$, so $O(E \log V)$.
Output condition: if `cnt==n`, `total` is MST weight; otherwise graph disconnected.

13. Shortest Paths

Chooser

unweighted BFS
DAG topo DP
weights ≥ 0 Dijkstra
negative edges Bellman-Ford

Relaxation

For edge (u, v, w) :

$\text{dist}[u] \neq \text{INF} \wedge \text{dist}[v] > \text{dist}[u] + w \Rightarrow \text{dist}[v] = \text{dist}[u] + w, \quad \text{parent}[v] = u.$

Dijkstra

```
fn dijkstra(adj:WGraph, source:usize) { // all weights >= 0
    let n:=adj.len();
    let mut dist:=vec![INF;n];
    let mut parent:=vec![None;n];
    let mut heap:=BinaryHeap::<(i32,usize)>::new();
    dist[source]=0;
    heap.push((0,source));
    while let Some((du,u))=heap.pop(){
        if du>dist[u] {continue} // stale heap entry
        for (v,w) in adj[u] {
            if dist[u]+w<dist[v] {
                dist[v]=du+w;
                parent[v]=Some(u);
                heap.push((dist[v],v));
            }
        }
    }
    dist
}
```

Runtime $O((V + E) \log V)$. Requires all weights ≥ 0 ; negative edges break it.
Bellman-Ford

Input `edges: Edgelist`. Negative edges OK; detects reachable negative cycles.
fn bellman_ford(n, edges, source){
 dist:=vec![INF;n]; parent:=vec![None;n]; dist[source]=0;
 for _ in 0..=n-1 {changed:=false;
 for e in edges {
 if dist[e.u]==INF && dist[e.v]>dist[e.u]+e.w {
 dist[e.v]=dist[e.u]+e.w; parent[e.v]=Some(e.u);
 changed=true;
 }
 if !changed {break}
 }
 for e in edges {
 if dist[e.u]==INF && dist[e.v]>dist[e.u]+e.w {
 return None; // reachable negative cycle
 }
 }
 }
 Some(dist)
}

Runtime $\Theta(V E)$. Returns `None` iff a reachable negative cycle is detected.
Negative cycle not reachable from source: Bellman-Ford still returns shortest paths. Only improvements after pass V reachable from source matter.

Shortest Paths in DAG

```
fn dag_shortest(adj:WGraph, source:usize){
    let n:=adj.len();
    let mut dist:=vec![INF;n];
    let mut parent:=vec![None;n];
    dist[source]=0;
    for u in topo(adj) {
        if dist[u]==INF {continue}
        for (v,w) in adj[u] { // topological order.
            if dist[v]>dist[u]+w { // one relaxation per edge
                dist[v]=dist[u]+w; // negative weights ok
                parent[v]=Some(u);
            }
        }
        dist
    }
}
```

Runtime $\Theta(V + E)$.

14. Flow Networks

Directed $G = (V, E)$, source s , sink t , capacities $c(u, v) \geq 0$.
 $0 \leq f(u, v) \leq c(u, v), \quad \forall u \neq s, t : \sum_v f(v, u) = \sum_u f(u, v),$
 $|f| = \sum_v f(v, s) - \sum_v f(t, v).$

Augmenting path: $s \rightarrow t$ path in residual graph; bottleneck = $\min c_f(e)$ on path.

Flow reduction recipe: 1. What is being selected? 2. Capacity enforcing usage limit. 3. Source/sink edges enforce constraints. 4. Flow value = objective. 5. Integral capacities => integral solution.

Ford-Fulkerson / Edmonds-Karp

Types: `cap[u][v]` residual capacity matrix; `adj` Graph stores forward+reverse residual neighbors.

```
fn max_flow(source:usize, sink:usize){ // max flow
    adj:=push(u); adj[v].push(u); cap[u][v]=c
    loop {
        pushed:=bfs_path(source,sink);
        parent.fill(None); parent[source]=Some(source);
        queue.push_back(source,INF);
        while let Some((u,flow))=queue.pop_front(){
            for v in adj[u] {
                if parent[v].is_none() && cap[u][v]>0 {
                    parent[v]=Some(u);
                    cap[u][v]-=pushed; cap[v][u]+=pushed;
                    pushed=flow.min(cap[u][v]);
                    if v==sink {return pushed}
                }
                queue.push_back((v,pushed))
            }
        }
    }
}
```

Ford-Fulkerson iterates caps: $O(E f^*)$. Edmonds-Karp BFS: $O(V E^2)$. Final residual graph gives min cut by residual reachability from s .

Max-Flow Min-Cut / Reductions

$\max |f| = \min_{s \in S, t \in T} c(S, V \setminus S).$

Optimality certificate: flow value F and cut capacity F .
Bipartite matching: source $\rightarrow L$ cap 1; $L \rightarrow R$ cap 1 for original edges; $R \rightarrow$ sink cap 1. Flow-1 $L \rightarrow R$ edges are matched pairs.

```
source=0; left=0; right=n-1; sink=n-1; left+n, right;
for i in 0..n {left {add edge(left,i,1)}
for i in 0..n {right {add edge(right+i,sink,1)}
for r in 0..n {right {add edge(right+r,sink,1)}
// flow-1 edge left+1 -> right+r = matched pair

Edge-disjoint paths: all edges cap 1. Vertex-disjoint: split v into v_in -> v_out cap 1; original u -> v becomes v_out -> v_in cap INF
```

15. Randomized / Probabilistic Analysis

Indicator: $X_A = 1[A], \mathbb{E}[X_A] = \Pr(A)$. Linearity:
 $\mathbb{E}\left[\sum_i X_i\right] = \sum_i \mathbb{E}[X_i]$

No independence needed.

Hiring:

$\Pr(X_i = 1) = 1/i \Rightarrow \mathbb{E}[\#hires] = \sum_{i=1}^n 1/i = H_n = \Theta(\log n).$

Hat-check:

$\Pr(X_i = 1) = 1/n \Rightarrow \mathbb{E}[\#fixed] = \sum_{i=1}^n 1/n = 1.$

Randomized quicksort: random pivot avoids input-dependent bad order; expected $\Theta(n \log n)$, worst still $\Theta(n^2)$.

Birthday: $\Pr[\text{all different among } q \text{ into } m] = \prod_{i=0}^{q-1} (1-i/m) \approx e^{-q(q-1)/(2m)}$;
for 365 days, $q = 23$ gives match prob $> 1/2$. Integral bound for decreasing positive f :
 $\sum_{k=1}^n f(k) \leq \int_0^n f(x) dx.$

16. Online Algorithms / Competitive Analysis / Experts

Online: decide before future known. Competitive ratio:
 $ALG(I) \leq r \cdot OPT(I) + c.$

Ski rental: rent 1/day, buy B , unknown T . $OPT(T) = \min(T, B)$. Rent until cost B , then buy. Ratio 2.
Experts / Weighted Majority
 n experts, rounds $t = 1..T$. Expert i predicts; loss $\ell_{i,t} \in \{0, 1\}$. Weights $w_i = 1$ initially. Let $W = \sum_i w_i$.

```
for t in 1..T {
    predict: weighted majority of experts;
    observe true outcome;
    for expert i wrong this round {
        w[i] *= beta; // 0 < beta < 1, often beta=2/3
    }
}
```

Weighted prediction: choose label with larger total weight. Potential: $W_t = \sum_i w_i^{(t)}$. If ALG makes mistake, wrong-weight $\geq W_t/2$, so $W_{t+1} \leq W_t/2 \cdot \beta + W_t/2 = W_t(1 + \beta)/2$. If best expert has m^* mistakes, its weight is β^{m^*} , hence $\beta^{m^*} \leq W_T \leq n((1 + \beta)/2)^M$. For $\beta = 1/2$: $M \leq 2.41(m^* + \log_2 n)$. General randomized Hedge: choose expert with prob w_i/W ; update $w_i \leftarrow w_i e^{-\eta \ell_{i,t}}$; regret $O(\sqrt{T \log n})$. Secretary: random order; skip first n/e , take next candidate better than all seen; success probability $\approx 1/e$. Prophet: online threshold vs prophet seeing all values.